Security Architectures in Machine Learning: A Comprehensive Analysis of Model Serialization Vulnerabilities

Author: Nirvana Fanaelahi

01 February 2026

Table of Contents

1. Introduction: The Epistemological Shift in AI Security

The domain of Computer Vision (CV) has historically operated under a paradigmatic assumption: that the security of a machine learning system is defined by its robustness against adversarial inputs or its ability to preserve the privacy of its training data. For decades, the curriculum for a Master's degree in Computer Vision focused on gradient descent, convolutional architectures, and the mathematics of optimization. Security, when discussed, was relegated to the study of adversarial perturbations pixel-level noise designed to fool a classifier or model inversion attacks aimed at reconstructing training images. However, the operational reality of modern Artificial Intelligence (AI) has exposed a far more fundamental and systemic vulnerability, one that resides not in the pixel space, but in the very infrastructure of model storage and transmission.

This report addresses the "AI Supply Chain" threat landscape, specifically the risks associated with model serialization. In the current ecosystem, pre-trained models are the currency of research and deployment. A student or engineer rarely trains a ResNet-50 or a Vision Transformer (ViT) from scratch; instead, they download "weights" from a central repository like Hugging Face, GitHub, or PyTorch Hub. This practice relies on a dangerous conflation of data and code. To a standard file system, a model checkpoint file (such as a .pth or .pkl file) appears to be a static binary blob. To the Python interpreter, however, it is often a paused program state, a sequence of instructions waiting to be resumed.

The implications of this "Model as Code" paradigm are profound. When a researcher loads a model file to inspect its architecture or run inference, they are potentially handing over control of their computing environment

to the author of that file. The mechanisms that allow for the flexible saving of complex Python objects specifically the pickle module used in PyTorch and older NumPy workflows are Turing-complete execution engines. They allow for the execution of arbitrary code (ACE) during the deserialization process, often without any explicit "run" command from the user.

This document serves as a comprehensive technical guide for advanced practitioners and students. It dissects the internal architectures of common serialization formats (.pth, .npz, .h5, .onnx, and .safetensors), analyzes the specific parameters that govern their security posture (such as weights_only and allow_pickle), and details the historical and emerging vulnerabilities that have plagued them, including the critical 2026 PyTorch weights_only bypass (CVE-2026-24747). By understanding these mechanisms, future architects of AI systems can adopt "Secure by Design" principles, moving away from convenience-oriented serialization toward strictly typed, data-centric storage formats.

2. The Theoretical Foundation: Serialization and the Pickle Protocol

To understand the specific vulnerabilities in PyTorch or NumPy, one must first master the underlying protocol that powers them: Python's pickle module. While often described simply as a "serialization format," pickle is more accurately understood as a stack-based programming language.

2.1 The Pickle Virtual Machine (PVM)

Unlike data interchange formats like JSON (JavaScript Object Notation)

or CSV (Comma-Separated Values), which are declarative and text-based, pickle is an imperative binary protocol. It does not describe *what* the data is; it describes *how* to construct it. When a Python program imports the pickle module and calls load(), it instantiates a Pickle Virtual Machine (PVM). This PVM reads a stream of opcodes (operation codes) from the file and executes them sequentially to manipulate a stack and a memo (a memory store for tracking objects).

The PVM supports a rich instruction set designed to reconstruct complex object graphs. These instructions include:

- **GLOBAL**: This opcode instructs the PVM to import a module and load a class or function from it. It takes two string arguments: the module name and the class/function name.
- **STACK_GLOBAL**: Similar to GLOBAL, but reads the names from the stack.
- **CALL**: This opcode calls a callable object (a function or a class constructor) that is currently on the stack, using arguments also present on the stack.
- **SETITEM**: This opcode takes a dictionary and a key-value pair from the stack and assigns the value to the key.
- **BUILD**: This opcode calls the __setstate__ method of an object to populate its internal state.

The critical realization for security is that the PVM does not distinguish between "safe" instructions (like creating an integer) and "dangerous" instructions (like importing the os module). It simply executes the opcode stream it is given.

2.2 The __reduce__ Mechanism and Arbitrary Code Execution

The primary vector for exploiting pickle and by extension, any format that

relies on it is the __reduce__ method. This method is part of Python's object-oriented extension interface, allowing custom objects to define how they should be serialized.

When the pickle module encounters an object, it checks for the existence of __reduce__. If present, the method is expected to return a tuple containing two elements:

1. A callable object (e.g., a function or a class).
2. A tuple of arguments to be passed to that callable.

During deserialization (loading), the PVM takes the callable and the arguments and executes callable(*arguments) to reconstruct the object.

The Vulnerability: An attacker can craft a malicious class where __reduce__ returns a pointer to a system function, such as os.system or subprocess.Popen, along with a shell command as the argument.

```
import os
import pickle

class MaliciousExploit:
    def __reduce__(self):
        # The PVM will execute this command immediately upon
deserialization
        return (os.system, ("rm -rf / --no-preserve-root",))
```

When a user loads a file containing an instance of MaliciousExploit, the PVM encounters the REDUCE opcode. It faithfully executes os.system("rm -rf /...") with the user's privileges. This execution happens *during* the loading process, often deep inside the library code (e.g., inside torch.load), long before the function returns the model object to the user.

This "exploit-on-load" characteristic makes inspecting a pickle file by loading it inherently unsafe.

2.3 The Persistence of Pickle in Computer Vision

Despite these known risks, pickle became the de facto standard for Python-based Machine Learning for several reasons related to research velocity and flexibility:

1. **Complex Object Graphs**: Deep learning models are not just matrices of numbers. They are complex hierarchies of Python classes (nn.Module, Optimizer, Scheduler) with shared references (e.g., tied weights in autoencoders). Pickle handles cyclic references and object identity automatically.
2. **Ease of Use**: The ability to save *any* Python object with torch.save(obj) without writing a schema or defining a protocol buffer definition allowed researchers to iterate rapidly.
3. **Legacy Integration**: Libraries like Scikit-learn and older versions of NumPy relied heavily on pickle, creating an ecosystem dependency.

However, this flexibility is technically "technical debt" accrued against security. The very feature that allows PyTorch to save arbitrary custom layers without configuration (the ability to serialize code structures) is the same feature that allows attackers to inject malware.

3. PyTorch Architecture: The .pth Ecosystem

PyTorch, developed by Meta AI, is the dominant framework in academic Computer Vision research. Its native file format, typically denoted by the extension .pth or .pt, is a container format that relies heavily on pickle.

3.1 Internal Anatomy of a .pth File

Since PyTorch version 1.6, the torch.save function has produced a **ZIP64 archive** by default. This change was made to unify the serialization format with TorchScript and to allow for better handling of large files. However, being a Zip file does not make it secure; it merely changes the container.
A typical .pth archive contains the following internal structure:

1. **data.pkl**: This is the "master" file. It contains the serialized Python object hierarchy (the dictionary of layers, the model class structure) encoded using the pickle protocol. However, the actual heavy tensor data is not stored inside this pickle stream.
2. **data/ Directory**: This internal folder contains a series of binary files (often named simply 0, 1, 2, etc.). These files hold the raw storage bytes for the tensors.
3. **version**: A simple text file indicating the serialization version number.

When a user calls torch.load("model.pth"):

1. PyTorch opens the Zip archive.
2. It creates an Unpickler object to process data.pkl.
3. The PVM executes the opcodes in data.pkl. When it encounters a tensor, it does not find the data inline; instead, it finds a "Persistent ID" pointing to one of the binary files in the data/ directory.
4. The unpickler reconstructs the tensor object in memory and links it to the raw data loaded from the binary file.

3.2 The Security Parameter: weights_only=True

For years, the standard advice for securing PyTorch models was "don't load models from untrusted sources." Recognizing that this was impractical in an era of open-source model hubs, PyTorch introduced the weights_only=True parameter in the torch.load function.

The Design Intent:
The weights_only=True parameter was designed to instantiate a restricted Unpickler. Unlike the standard pickle.Unpickler, which allows importing any global module, this restricted version was whitelisted to allow only a specific set of safe types:

- Primitive types (integers, floats, strings, bytes).
- Container types (lists, tuples, dictionaries, sets).
- PyTorch-specific types (torch.Tensor, torch.nn.Parameter, OrderedDict).
- Data types (NumPy scalars).

Crucially, it was designed to block the GLOBAL opcode from importing system modules like os, sys, or subprocess. If the PVM encountered GLOBAL os.system, the restricted unpickler would raise an UnpicklingError.

3.3 The Failure of Mitigation: CVE-2026-24747

In early 2026, the fragility of attempting to secure a Turing-complete protocol via allowlisting was demonstrated by a high-severity vulnerability: **CVE-2026-24747**.
Technical Mechanics of the Exploit:
Security researchers discovered that the restricted unpickler, while successfully blocking explicit imports of dangerous functions, failed to validate the arguments of certain memory-management opcodes, specifically SETITEM and SETITEMS.
In a standard pickle stream, SETITEM is used to assign a value to a key in a dictionary. However, the researchers found that if SETITEM was applied to a non-dictionary object (specifically, a crafted PyTorch storage object) in a specific sequence, it could trigger a heap memory corruption.
The exploit involved:

1. **Heap Grooming**: Using a sequence of valid, allowlisted opcodes

(like creating lists and tensors) to arrange the Python process's heap memory into a predictable state.

2. **Corruption**: Using the malformed SETITEM call to write data past the bounds of an allocated object.
3. **Control Flow Hijacking**: Overwriting a function pointer in memory (e.g., a destructor or a virtual function table) with the address of a "gadget" (a sequence of assembly instructions already present in the process memory, such as system()).

This vulnerability allowed attackers to achieve Remote Code Execution (RCE) *even when* weights_only=True was enabled. The restricted unpickler checked the *names* of the functions being called (blocking os.system) but could not check the *side effects* of valid memory operations.

Implications for Students:

This incident serves as a critical lesson: **Parsers for complex formats are inherently attack surfaces.** A flag like weights_only=True is a heuristic defense; it reduces the attack surface but does not eliminate the fundamental risk of processing complex instructions from an untrusted source.

3.4 Zip-Based Attack Vectors

Because modern .pth files are Zip archives, they also inherit the vulnerabilities associated with Zip processing. Two notable vectors are:

1. **Zip-Slip (Directory Traversal)**: A malicious Zip archive can contain files with names like ../../../../etc/passwd or ../../.bashrc. If the extraction logic in the model loader does not sanitize these paths, loading the model could overwrite critical system files on the user's machine. While Python's zipfile module has warnings about this, custom loading logic in some academic codebases may be vulnerable.

2. **The Hidden Pickle (GHSA-769v-p64c-89pr)**: Security scanners like picklescan (discussed later) often scan files based on extensions (checking data.pkl). Attackers developed a technique to hide a malicious pickle file inside the archive with a benign extension (e.g., config.txt or metadata.bin). They would then modify the main data.pkl to include a legitimate instruction to load this secondary file using torch.load. The scanner would approve the main file (which looks safe) and ignore the secondary file (wrong extension), but the PVM would execute the payload when the secondary file was loaded during the reconstruction process.

4. NumPy Architecture: .npz and .npy

While PyTorch models define the architecture and weights, Computer Vision workflows heavily utilize NumPy for dataset management (e.g., storing images, labels, and bounding boxes). The security of these files revolves around the format specifications and the allow_pickle parameter.

4.1 Format Specifications

The .npy Format:

The .npy format is designed to store a single NumPy array. It consists of:

1. **Magic String**: \x93NUMPY followed by a version byte.
2. **Header**: A text-based dictionary describing the array's format. This includes:
    - descr: The data type descriptor (e.g., <f8 for little-endian 64-bit float).
    - fortran_order: Boolean indicating memory layout.
    - shape: A tuple indicating dimensions.

3. **Data**: The binary bytes of the array data.

The .npz Format:

The .npz format is simply an uncompressed Zip archive containing multiple .npy files. When you load an .npz, you get a dictionary-like object where keys are filenames and values are the arrays.

4.2 The allow_pickle Parameter

The security of NumPy loading is binary: it is either completely safe (for numerical data) or completely unsafe (for object data). This is controlled by the allow_pickle argument in numpy.load().

Historical Context:

Prior to NumPy version 1.16.3, the default value for allow_pickle was True. This meant that np.load() would automatically try to unpickle data if it couldn't interpret it as raw numbers. This was convenient for saving arrays of strings or generic Python objects but disastrous for security. Following the disclosure of CVE-2019-6446, the default was changed to False.

The allow_pickle=True Risk:

In Computer Vision, datasets often contain non-numeric data, such as:

- File paths (strings).
- Variable-length lists of bounding boxes (object arrays).
- Dictionaries of metadata.

If a researcher saves such a dataset, NumPy forces them to use object serialization (pickling). Consequently, when a student downloads this dataset, they must run np.load(filename, allow_pickle=True) to read it.

Once allow_pickle=True is set, the vulnerability mechanism is identical to the one described in the PyTorch section. The .npy header can specify a dtype of object, and the binary data following it is a pickle stream. The PVM is instantiated, and any __reduce__ payload embedded in the stream is executed.

Best Practice for Students:
Students must be taught that allow_pickle=True is effectively a "Run Executable" permission. If a dataset requires this flag, it should be treated with the same suspicion as an .exe or .sh file.

4.3 Buffer Overflows in C Parsers

Even with allow_pickle=False, the .npy format is parsed by C code for performance. This introduces the risk of traditional binary exploitation, specifically buffer overflows and integer overflows.
Attackers can craft .npy headers that claim to have an extremely large shape (e.g., (2**64, 2**64)). If the parser attempts to calculate the memory requirement by multiplying these dimensions without checking for integer overflow, it might allocate a small buffer but attempt to read a large amount of data into it. This "Heap Buffer Overflow" can potentially lead to code execution, though it is significantly harder to exploit than the "by design" RCE of pickle.

5. The Secure Alternative: Safetensors

The industry response to the systemic insecurity of Pickle-based formats has been a shift toward "Safe by Design" architectures. The leading standard in this new paradigm is **Safetensors**, developed by Hugging Face and now widely adopted across the AI ecosystem (including Stability AI and EleutherAI).

5.1 Architectural Philosophy: Data, Not Code

The core philosophy of Safetensors is the strict separation of **Model Definition** (Code) and **Model Weights** (Data). A Safetensors file cannot

store a Python class definition or a function call. It can only store tensors. This limitation is its primary security feature.

Internal Structure:

A .safetensors file consists of two contiguous blocks:

1. **Header Block (N bytes)**: A UTF-8 encoded JSON string. This header serves as a directory for the file. It is a dictionary where keys are tensor names (e.g., transformer.h.0.attn.c_attn.weight) and values are metadata objects describing:
    - dtype: Data type (e.g., "F32", "F16", "BF16").
    - shape: A list of dimensions.
    - data_offsets: A tuple [start, end] indicating the byte range in the data block.
2. **Data Block**: A flat binary sequence containing the raw tensor data for all tensors, concatenated together.

5.2 Security Guarantees

1. Zero Arbitrary Code Execution (RCE):

Because the header is JSON and the body is raw bytes, there is no virtual machine involved in loading a Safetensors file. The loader parses the JSON (a safe, declarative format) and then reads bytes from the specified offsets. There is no mechanism to express "import os" or "call function" within the file format. Therefore, the class of attacks relying on __reduce__ or PVM exploitation is architecturally impossible.

2. Denial of Service (DoS) Prevention:

The format specification enforces a limit on the size of the JSON header (typically 100MB). This prevents "Billion Laughs" style attacks or memory exhaustion attacks where a malicious file attempts to force the parser to allocate terabytes of RAM for the metadata.

5.3 Performance Advantages: Memory Mapping (mmap)

Beyond security, Safetensors offers significant performance advantages that are particularly relevant for the large models used in modern CV (like Vision Transformers or Stable Diffusion).

The Mechanism of mmap:

In a traditional Pickle load, the PVM must read the entire file stream sequentially to reconstruct the object state in memory. This involves copying data from the disk cache to the user space buffer, often resulting in a memory spike of 2x the model size during loading (one copy on disk, one in RAM).

Safetensors exploits the OS-level mmap system call. Because the JSON header provides the exact byte offsets of every tensor *before* any data is read, the loader can map the file directly into the process's virtual address space.

- **Lazy Loading**: If a user only needs the weights for layer1, the OS only pages in the specific bytes for layer1 from the disk. The rest of the model remains on disk.
- **Zero-Copy**: The data is available in memory without explicit copying by the CPU, drastically reducing loading time and CPU overhead.

5.4 Migration Guide for Students

The transition from .pth to .safetensors requires minor changes to the student's workflow.

Saving a Model:

- *Old (PyTorch)*: torch.save(model.state_dict(), "model.pth")
- *New (Safetensors)*:

Python
from safetensors.torch import save_file
save_file(model.state_dict(), "model.safetensors")

Loading a Model:

- *Old (PyTorch)*:
  Python
  state_dict = torch.load("model.pth") # Insecure point
  model.load_state_dict(state_dict)

- *New (Safetensors)*:
  Python
  from safetensors.torch import load_file
  state_dict = load_file("model.safetensors") # Safe
  model.load_state_dict(state_dict)

Pedagogical Note: Students will notice that they cannot save the *entire* model object (including the class definition) with Safetensors, only the state_dict. This is intentional. It forces the code (architecture) to be version-controlled in .py files and the data (weights) to be version-controlled in .safetensors files, preventing the "Hidden Code" vulnerability where a .pth file silently redefines the model architecture to something malicious.

6. The Broader Ecosystem: ONNX, Keras, and GGUF

While PyTorch and Safetensors are the primary focus, CV practitioners will encounter other formats in the wild.

6.1 ONNX (Open Neural Network Exchange)

ONNX is a graph-based format designed for interoperability. It represents the model as a graph of nodes (operators) and edges (tensors).

Security Profile:

- **Graph-Based Safety**: ONNX is generally safer than Pickle because it does not serialize arbitrary Python objects. It serializes a computation graph using Protocol Buffers (protobuf).
- **The Custom Operator Risk**: To support novel research, ONNX allows for "Custom Operators." These are nodes in the graph that do not correspond to standard operations (like Conv2D) but instead refer to a named function that must be implemented by the runtime. If a student loads a model with a malicious custom operator, and their environment happens to have a corresponding implementation (or acts on it), it could trigger unsafe behavior.
- **Metadata Injection (2025 Findings)**: Recent research has identified that while ONNX itself doesn't execute code, its metadata_props field can be abused. Attackers can inject Base64-encoded Python payloads into these metadata fields. While the ONNX runtime ignores them, helper scripts or pipelines that "inspect" models might extract and execute this metadata (e.g., via eval()), creating a vulnerability in the *tooling* rather than the format itself.

6.2 Keras (.h5 and .keras)

Keras (TensorFlow) models have historically used HDF5 (.h5) and recently moved to a new Zip-based .keras format.

The Lambda Layer Vulnerability:

Keras allows for "Lambda Layers"—layers defined by arbitrary Python

functions. These are serialized as Python bytecode (using marshal or pickle) inside the model file.

- **Exploit**: An attacker can embed a malicious Lambda layer that executes system commands during the model building phase.
- **Mitigation (safe_mode)**: Keras v3 introduced a safe_mode=True parameter for loading. This disables the deserialization of Lambda layers. However, similar to the PyTorch weights_only bypass, researchers have found ways to construct "gadget chains" within the allowed Keras config deserialization to achieve code execution, though it is more difficult than with raw pickle.

6.3 GGUF (GPT-Generated Unified Format)

Popularized by the llama.cpp community for running Large Language Models (LLMs) on consumer hardware, GGUF is a binary format similar in spirit to Safetensors but with more complex metadata for quantization.

Security Profile:
GGUF is generally safe from RCE but vulnerable to **Memory Corruption** attacks. Because GGUF parsers are typically written in C++ (for performance), they are susceptible to:

- **Integer Overflows**: Manipulating the key-value count (n_kv) in the header to trigger incorrect memory allocation.
- **Buffer Overflows**: Providing data chunks larger than the declared size.
- **Denial of Service**: Malformed headers causing infinite loops in the parser.
  Unlike Pickle, these are "bugs" in the parser, not features of the format.

7. Supply Chain Defense Strategies

Understanding file formats is the first step. The second is defending the workflow.

7.1 Static Analysis: Scanning

Tools like picklescan (by Hugging Face) and ModelScan (by Protect AI) are designed to "x-ray" model files before they are loaded.

Mechanism:

These tools parse the pickle stream without executing it. They simulate the stack operations and check if any GLOBAL or REDUCE opcodes reference blocklisted modules (like os, sys, subprocess, shutil).

Limitations and Bypasses:

- **Obfuscation**: Attackers can use complex import chains (e.g., importing torch and using torch.__builtins__ to access os) to fool the scanner.
- **Zip Hiding**: As mentioned in the PyTorch section, attackers can hide the malicious pickle in a secondary file inside the Zip archive that the scanner ignores because it doesn't end in .pkl.
- **The Arms Race**: Scanning is heuristic. It will always be one step behind novel exploitation techniques. It should be used as a "smoke detector," not a "firewall."

7.2 Dynamic Analysis: Sandboxing

For a Master's student replicating a paper, the only robust way to run an untrusted .pth model is isolation.

- **Network Isolation**: Run the inference script in a Docker container with --network none. This prevents the malware from exfiltrating credentials (SSH keys, AWS tokens) or downloading second-stage

payloads.

- **Filesystem Isolation**: Mount the model directory as Read-Only.
- **Ephemeral Environments**: Use temporary VMs (like Google Colab instances or throwaway cloud instances) rather than a personal laptop or a university cluster head node.

7.3 Cryptographic Verification

Trust must be rooted in identity.

- **SHA256 Verification**: When downloading a model (e.g., resnet50.pth), students should habitually verify the file's hash against the official release page. A mismatch indicates tampering (or corruption).
  - *Command*: sha256sum model.pth
- **Digital Signatures**: The industry is adopting tools like Sigstore to digitally sign models. A signature proves that the file originated from a specific entity (e.g., Meta or Google) and hasn't been modified. While this doesn't guarantee the model is benign (the entity could be malicious), it establishes accountability.

8. Comparative Analysis of Serialization Formats

The following tables synthesize the security profiles of the formats discussed.

Table 1: Security Risk Profile by Format

Format	Extension	Primary Security Risk	Exploitation Mechanism	Security Rating

PyTorch (Pickle)	.pth, .pt	**Critical (RCE)**	PVM __reduce__ exploit; Heap corruption	**Unsafe** (Avoid for sharing)
NumPy (Pickle)	.npy, .npz	**High** (if allow_pickle=True)	PVM __reduce__ exploit	**Conditionally Unsafe**
NumPy (No Pickle)	.npy	**Low**	Parser Buffer/Integer Overflows	**Safe**
Safetensors	.safetensors	**Negligible**	None (Zero Code Execution)	**Secure (Gold Standard)**
ONNX	.onnx	**Low**	Custom Ops / Parser bugs	**Generally Safe**
Keras	.h5, .keras	**Medium**	Lambda Layers / Gadget Chains	**Medium** (Use safe_mode)

Table 2: Parameter Safety Reference

Library	Function	Parameter	Safe Value	Unsafe Value	Implication of Unsafe Value
PyTorch	torch.load	weights_only	True*	False	Enables full PVM; allows os.system. *(Note: True can still be bypassed via CVE-2026-24747)*
NumPy	np.load	allow_pickle	False	True	Enables PVM; allows os.system.
Keras	load_model	safe_mode	True	False	Allows execution of embedded Lambda layer code.
Hugging Face	hf_hub_download	verify_hash	True	False	Disables integrity check; allows Man-in-

					the-Middle tampering.

9. Conclusion and Future Outlook

The landscape of AI security has shifted. The threat is no longer just the "adversarial example" that tricks a car into seeing a stop sign as a speed limit; it is the "poisoned model" that tricks the researcher into executing a remote shell.

For the Master's student in Computer Vision, the key takeaways are:

1. **Code vs. Data**: Always distinguish between the model architecture (which should be inspectable Python code) and the model weights (which should be inert data).
2. **Abandon Pickle**: The convenience of torch.save(model) is a security liability. Adopt safetensors for all model storage and distribution.
3. **Trust No One**: Treat every .pth and .npz file from the internet as potentially hostile code. Isolate it, scan it, and never load it with privileges you aren't willing to lose.

As we move forward, we expect regulatory frameworks and library defaults to become stricter PyTorch is already moving to make weights_only=True the default. However, the legacy of millions of insecure .pth files will remain in the ecosystem for years. Navigating this "brownfield" requires not just tools, but the vigilance and architectural understanding detailed in this report.